Project Report

INVESTMENT PORTFOLIO AND RISK MANAGEMENT SYSTEM

Prepared by: Akanksha Prashant Kumar Kulkarni

1. Abstract

The Investment Portfolio and Risk Management System is a robust, database-driven application crafted to support individual investors in effectively managing and analyzing their financial investments. In an increasingly complex and fast-paced financial environment, investors require a dependable system that allows them to track various investment portfolios and assets, record transactions seamlessly, and monitor associated investment risks.

This project directly addresses these essential needs by implementing a structured and user-friendly full-stack application with a React-based frontend, a Node.js/Express backend, and a PostgreSQL relational database. The system is connected via API-based DB connectivity using the pg library, exposing RESTful endpoints that allow full CRUD operations on all entities.

Key functionalities include investor management with risk profiling, portfolio creation and valuation, asset and asset class classification, investment tracking, real-time market data monitoring, transaction history logging, and automated risk assessment. The system employs BCNF-normalized database tables ensuring minimal redundancy and maximum data integrity, while the backend enforces referential integrity through foreign key constraints and input validation at the API layer.

Component	Technology Used
Frontend	HTML, CSS, JavaScript / React
Backend	Node.js with Express.js
Database	PostgreSQL
Connectivity	API-based DB Connectivity (REST API via pg library)

2. Introduction

2.1 Background of the Problem

Modern investors manage multiple asset classes — equities, bonds, gold, real estate, and cryptocurrencies — simultaneously. Tracking the performance, risk, and transactions of these diverse portfolios manually is error-prone, time-consuming, and inefficient. Traditional spreadsheet-based systems lack real-time updating, relational data integrity, and structured query capabilities.

2.2 Need for Database Connectivity

A persistent, normalized relational database is essential to store structured financial data with referential integrity. Database connectivity through REST APIs allows a modern web frontend to interact securely and efficiently with the database, enabling real-time CRUD operations, complex joins for reporting, and scalable data management without direct database access from the client.

2.3 Objective of the Project

- Design and implement a fully normalized PostgreSQL database (BCNF) for the Portfolio Management domain.
- Build a RESTful Node.js/Express backend that connects to the database via the pg library (API-based connectivity).
- Develop a React frontend that communicates with the backend through HTTP requests.
- Implement full CRUD operations for all entities: Investors, Portfolios, Assets, Investments, Transactions, Market Data, and Risk Assessments.
- Enforce input validation, error handling, and role-based access control.
- Demonstrate real-world database concepts: ER modeling, normalization to BCNF, primary/foreign key constraints, referential integrity, and indexing.

3. System Overview

3.1 Description of the Application

The Investment Portfolio and Risk Management System is a full-stack web application that provides investors with a centralized platform to manage their financial activities. The system follows a three-tier architecture: a React-based presentation layer, a Node.js/Express application layer, and a PostgreSQL data layer. The frontend renders dynamic dashboards for portfolio performance, market prices, and risk indicators. The backend processes HTTP requests, executes parameterized SQL queries via a connection pool, and returns structured JSON responses.

3.2 Target Users

- Individual retail investors managing diversified portfolios.
- Financial advisors tracking multiple client accounts.
- Database System course students demonstrating full-stack DBMS implementation.
- Administrators performing system-wide data management and reporting.

3.3 Scope of the System

The system covers 8 core modules mapped to 8 normalized database tables. It supports investor onboarding with risk profiling, portfolio creation and management, asset catalogue maintenance, investment recording, transaction auditing, live market data tracking, and portfolio-level risk assessment. The backend exposes 35+ REST API endpoints. The system does not include real-time market data feeds from external APIs (data is entered manually or seeded), nor does it include payment processing.

4. Technologies Used

Layer	Technology	Role
Frontend	React.js + Axios	Dynamic UI components, API calls, state management
Frontend	HTML5 + CSS3	Page structure and responsive styling
Backend	Node.js v18+	Server-side JavaScript runtime environment
Backend	Express.js v4	HTTP routing, middleware, RESTful API framework
Database	PostgreSQL 15	Relational DBMS for persistent normalized data storage
DB Connectivity	pg (node-postgres)	PostgreSQL client — API-based DB connectivity via connection pool
Security	dotenv	Environment variable management for DB credentials
Security	bcrypt	Password hashing for user authentication
Security	JSON Web Token (JWT)	Stateless authentication and authorization
Dev Tools	Postman	API testing and endpoint validation
Dev Tools	pgAdmin 4	PostgreSQL GUI for database administration
Dev Tools	VS Code	Integrated development environment

5. Database Design

5.1 ER Diagram — Entity and Relationship Overview

The Entity-Relationship diagram below represents the conceptual design of the system. It illustrates 8 main entities, their attributes, and their relationships. The diagram follows Chen notation with rectangles for entities, ellipses for attributes, and diamonds for relationships.

ER RELATIONSHIP MAP

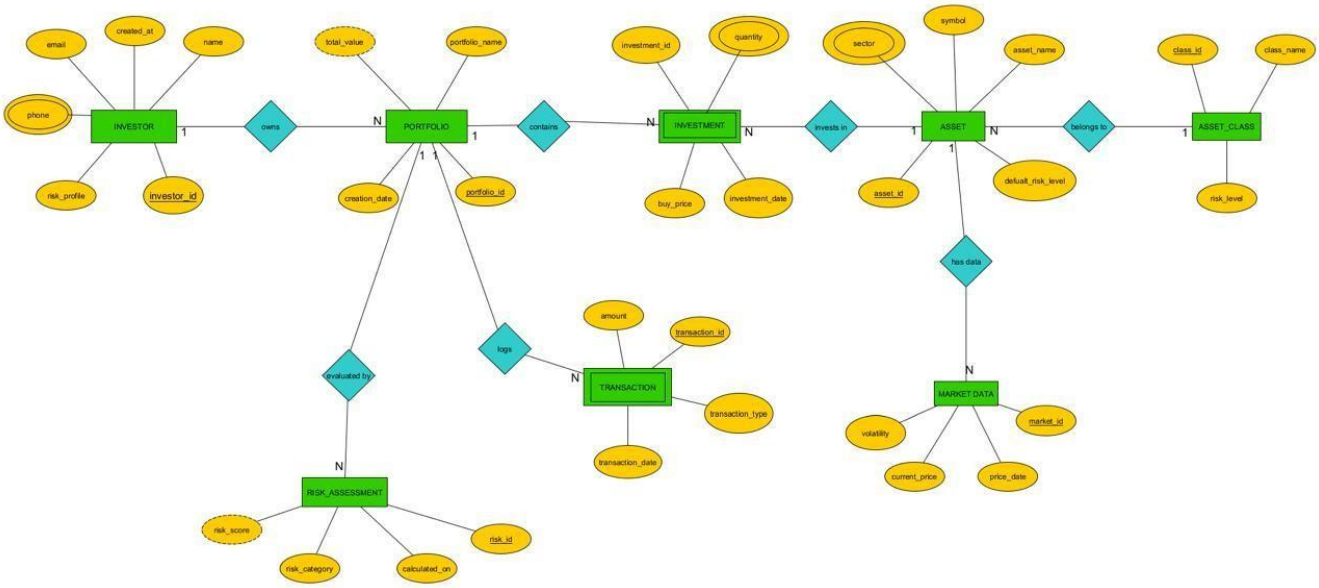
RISK_PROFILE ←(1:N)— INVESTOR ←(1:N)— PORTFOLIO

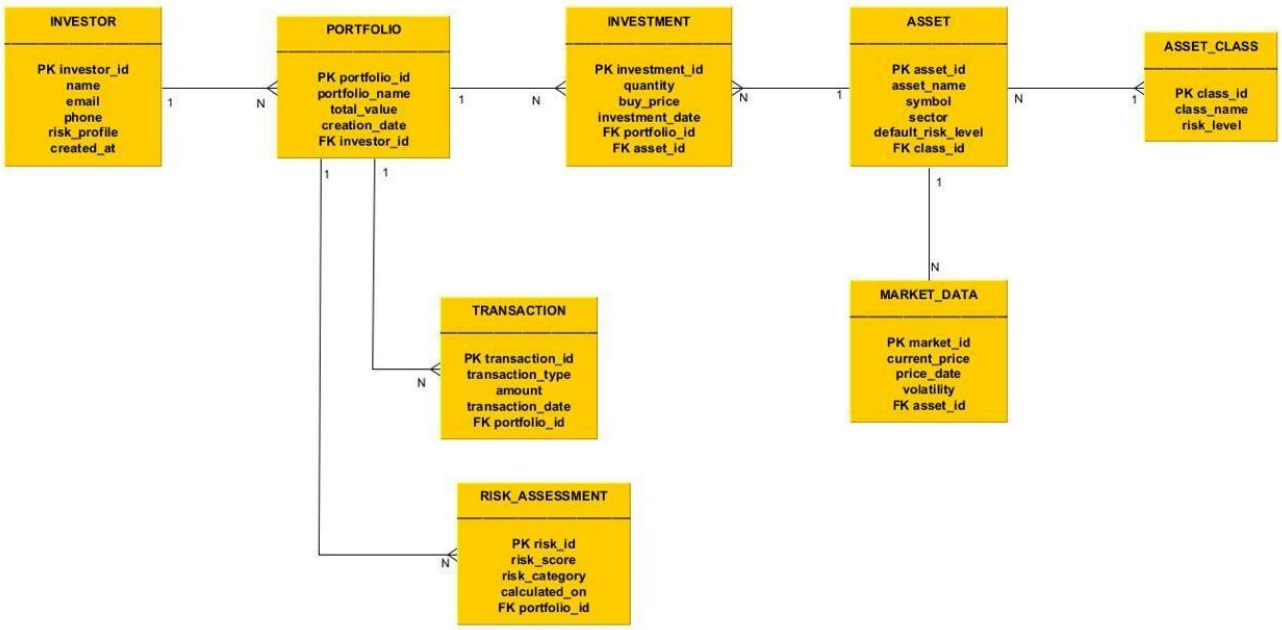
ASSET_CLASS ←(1:N)— ASSET ←(1:N)— INVESTMENT —(N:1)→ PORTFOLIO

ASSET ←(1:N)— TRANSACTION —(N:1)→ PORTFOLIO

ASSET ←(1:1)— MARKET_DATA

PORTFOLIO ←(1:N)— RISK_ASSESSMENT





The diagram captures the following key relationships:

- INVESTOR owns one or more PORTFOLIO records (1:N). Each portfolio belongs to exactly one investor.
- PORTFOLIO contains multiple INVESTMENT records (1:N). INVESTMENT is a bridge/junction table between PORTFOLIO and ASSET (M:N resolved).
- ASSET belongs to an ASSET_CLASS (N:1). Each asset is classified under exactly one asset class.
- ASSET has associated MARKET_DATA (1:1). Each asset has one current market data record.
- PORTFOLIO has multiple TRANSACTION records (1:N), logging every buy/sell/deposit/withdraw action.
- PORTFOLIO is evaluated by RISK_ASSESSMENT (1:N). Multiple risk evaluations can be performed over time.
- INVESTOR references RISK_PROFILE (N:1) to classify investor risk tolerance.

5.2 Relationship Cardinality Summary

Relationship	Cardinality	Foreign Key	Description
RISK_PROFILE → INVESTOR	One-to-Many (1:N)	risk_profile	One risk profile can apply to many investors
INVESTOR → PORTFOLIO	One-to-Many (1:N)	investor_id	One investor can own multiple portfolios
PORTFOLIO → INVESTMENT	One-to-Many (1:N)	portfolio_id	A portfolio holds multiple investments
ASSET → INVESTMENT	One-to-Many (1:N)	asset_id	An asset can appear in many investments
PORTFOLIO → TRANSACTION	One-to-Many (1:N)	portfolio_id	A portfolio can have many transactions
ASSET → TRANSACTION	One-to-Many (1:N)	asset_id	An asset can be part of many transactions
ASSET → MARKET_DATA	One-to-One (1:1)	asset_id	Each asset has one market data record
ASSET_CLASS → ASSET	One-to-Many (1:N)	class_id	One asset class groups multiple assets
PORTFOLIO → RISK_ASSESSMENT	One-to-Many (1:N)	portfolio_id	A portfolio can have multiple risk evaluations

5.3 Table Structure (All 8 Normalized Tables)

The following section details each table's structure including column names, data types, and constraints. All tables have been normalized to Boyce-Codd Normal Form (BCNF).

Table 1: RISK_PROFILE

Column Name	Data Type	Constraint	Description
risk_profile	VARCHAR(20)	PRIMARY KEY	Risk category: Low / Medium / High
risk_desc	TEXT	NOT NULL	Text description of risk tolerance

Table 2: INVESTOR

Column Name	Data Type	Constraint	Description
investor_id	SERIAL	PRIMARY KEY	Auto-increment unique investor ID
name	VARCHAR(100)	NOT NULL	Full name of the investor
email	VARCHAR(150)	UNIQUE, NOT NULL	Unique contact email address
phone	VARCHAR(15)	—	Contact phone number
risk_profile	VARCHAR(20)	FK → RISK_PROFILE	Investor's risk tolerance category
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

Table 3: PORTFOLIO

Column Name	Data Type	Constraint	Description
portfolio_id	SERIAL	PRIMARY KEY	Unique portfolio identifier
portfolio_name	VARCHAR(100)	NOT NULL	Name of the portfolio
total_value	NUMERIC(15,2)	DEFAULT 0	Aggregated value of all investments
creation_date	DATE	DEFAULT CURRENT_DATE	Date portfolio was created
investor_id	INT	FK → INVESTOR	Owner investor reference

Table 4: ASSET_CLASS

Column Name	Data Type	Constraint	Description
class_id	SERIAL	PRIMARY KEY	Unique asset class ID
class_name	VARCHAR(100)	NOT NULL	Name of the asset class (Equity, Debt, etc.)
risk_level	VARCHAR(20)	NOT NULL	Risk level of this class

Table 5: ASSET

Column Name	Data Type	Constraint	Description
asset_id	SERIAL	PRIMARY KEY	Unique asset identifier
asset_name	VARCHAR(100)	NOT NULL	Full name of the financial asset
symbol	VARCHAR(20)	NOT NULL	Trading ticker symbol
sector	VARCHAR(100)	—	Industry sector of the asset
default_risk_level	VARCHAR(20)	—	Asset-specific default risk level
class_id	INT	FK → ASSET_CLASS	Asset classification reference

Table 6: INVESTMENT

Column Name	Data Type	Constraint	Description
investment_id	SERIAL	PRIMARY KEY	Unique investment record ID
quantity	NUMERIC(10,4)	NOT NULL	Number of units purchased
purchase_price	NUMERIC(15,2)	NOT NULL	Price per unit at time of investment
investment_date	DATE	DEFAULT CURRENT_DATE	Date the investment was made
portfolio_id	INT	FK → PORTFOLIO	Portfolio that owns this investment
asset_id	INT	FK → ASSET	Asset that was invested in

Table 7: TRANSACTION

Column Name	Data Type	Constraint	Description
transaction_id	SERIAL	PRIMARY KEY	Unique transaction record ID
transaction_type	VARCHAR(10)	CHECK (BUY/SELL/DEPOSIT/WITHDRAW)	Type of financial transaction
amount	NUMERIC(15,2)	NOT NULL	Total transaction amount in INR
transaction_date	TIMESTAMP	DEFAULT NOW()	Date and time of transaction
portfolio_id	INT	FK → PORTFOLIO	Associated portfolio
asset_id	INT	FK → ASSET (nullable)	Asset involved (NULL for DEPOSIT/WITHDRAW)

Table 8: MARKET_DATA

Column Name	Data Type	Constraint	Description
market_id	SERIAL	PRIMARY KEY	Unique market data record ID
current_price	NUMERIC(15,2)	NOT NULL	Latest market price of the asset
last_updated	TIMESTAMP	DEFAULT NOW()	Last price update timestamp
volatility	NUMERIC(5,2)	—	Price fluctuation metric
asset_id	INT	FK → ASSET	Asset this market data belongs to

Table 9: RISK_ASSESSMENT

Column Name	Data Type	Constraint	Description
risk_id	SERIAL	PRIMARY KEY	Unique risk assessment record ID
risk_score	NUMERIC(5,2)	NOT NULL	Calculated numerical risk score (0-10)
risk_category	VARCHAR(20)	NOT NULL	Categorized level: Low / Medium / High
calculated_on	DATE	NOT NULL	Date risk was assessed
portfolio_id	INT	FK → PORTFOLIO	Portfolio being assessed

6. Database Connectivity Implementation (API-based)

6.1 Steps Involved

The system uses API-based database connectivity through the Node.js pg library, which implements a connection pool to PostgreSQL. The following steps are performed for every API request:

1. Step 1 — Load Driver: The pg (node-postgres) library is loaded as the PostgreSQL client driver using require('pg').
2. Step 2 — Establish Connection: A Pool object is created with credentials from .env (DB_USER, DB_HOST, DB_NAME, DB_PASSWORD, DB_PORT). The pool manages multiple concurrent connections.
3. Step 3 — Create Statement: Parameterized queries are prepared using template literals with $1, $2, ... placeholders to prevent SQL injection.
4. Step 4 — Execute Query: pool.query(sql, params) is called asynchronously using async/await. The query is sent to PostgreSQL over the established connection.
5. Step 5 — Process Result: The returned result.rows array is extracted and returned as JSON in the HTTP response.
6. Step 6 — Close Connection: The connection pool automatically manages release back to the pool. The pool itself persists for the server lifetime.

6.2 Database Connection Code (config/db.js):

```
// config/db.js
const { Pool } = require('pg');
const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: 5432,
});
pool.connect()
  .then(() => console.log('✅ PostgreSQL connected successfully'))
  .catch(err => console.error('❌ PostgreSQL connection error:', err));
module.exports = pool;
```

6.3 Express Server Entry Point (index.js):

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const cors = require('cors');
require('dotenv').config();
const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: {
    origin: process.env.FRONTEND_URL || 'http://localhost:5173',
    methods: ['GET', 'POST'],
  },
});
// Middleware
app.use(cors({ origin: process.env.FRONTEND_URL || 'http://localhost:5173' }));
app.use(express.json());
// Make io accessible to routes
app.set('io', io);
app.get('/', (req, res) => {
  res.send('✅ Portfolio Management Backend is running!');
});
// Routes
app.use('/auth', require('./routes/authRoutes'));
app.use('/investors', require('./routes/investorRoutes'));
app.use('/portfolios', require('./routes/portfolioRoutes'));
app.use('/assets', require('./routes/assetRoutes'));
app.use('/investments', require('./routes/investmentRoutes'));
app.use('/transactions', require('./routes/transactionRoutes'));
app.use('/market-data', require('./routes/marketRoutes'));
app.use('/reports', require('./routes/reportRoutes'));
// WebSocket
io.on('connection', (socket) => {
  console.log('Client connected:', socket.id);
  socket.on('disconnect', () => console.log('Client disconnected:', socket.id));
});
// Simulate live market updates every 5s
setInterval(() => {
```

```
    io.emit('market_update', {
      timestamp: new Date().toISOString(),
      data: {
        NIFTY: +(Math.random() * 200 + 22000).toFixed(2),
        SENSEX: +(Math.random() * 500 + 73000).toFixed(2),
        GOLD: +(Math.random() * 100 + 72000).toFixed(2),
      },
    });
  }, 5000);
  const PORT = process.env.PORT || 5000;
  server.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

6.4 Sample Model Query (models/investorModel.js):

```
const pool = require('../config/db');
const getAllInvestors = async () => {
  const result = await pool.query(`
    SELECT
      i.investor_id,
      i.name,
      i.email,
      i.phone,
      i.risk_profile,
      r.risk_desc,
      i.created_at
    FROM investor i
    LEFT JOIN risk_profile r ON i.risk_profile = r.risk_profile
    ORDER BY i.investor_id
  `);
  return result.rows;
};
const getInvestorById = async (id) => {
  const result = await pool.query(`
    SELECT
      i.investor_id,
      i.name,
      i.email,
      i.phone,
      i.risk_profile,
      r.risk_desc,
      i.created_at
    FROM investor i
    LEFT JOIN risk_profile r ON i.risk_profile = r.risk_profile
    WHERE i.investor_id = $1
  `, [id]);
  return result.rows[0];
};
const createInvestor = async ({ name, email, phone, risk_profile }) => {
  const result = await pool.query(`
    INSERT INTO investor (name, email, phone, risk_profile)
    VALUES ($1, $2, $3, $4)
```

```
    RETURNING *
  `, [name, email, phone, risk_profile]);
  return result.rows[0];
};
const getInvestorsByRisk = async (risk) => {
  const result = await pool.query(`
    SELECT
      i.investor_id,
      i.name,
      i.email,
      i.risk_profile,
      r.risk_desc
    FROM investor i
    LEFT JOIN risk_profile r ON i.risk_profile = r.risk_profile
    WHERE i.risk_profile = $1
    ORDER BY i.name
  `, [risk.toUpperCase()]);
  return result.rows;
};
module.exports = { getAllInvestors, getInvestorById, createInvestor, getInvestorsByRisk };
```

6.5 Sample Model Query (models/assetModel.js):

```
// models/assetModel.js
const pool = require('../config/db');
// Get all assets
const getAllAssets = async () => {
  const result = await pool.query(`
    SELECT
      a.asset_id,
      a.asset_name,
      a.symbol,
      a.sector,
      a.default_risk_level,
      a.class_id,
      ac.class_name,
      ac.risk_level AS class_risk_level
    FROM asset a
    LEFT JOIN asset_class ac ON a.class_id = ac.class_id
    ORDER BY a.asset_id
  `);
  return result.rows;
};
// Get single asset by ID
const getAssetById = async (id) => {
  const result = await pool.query(`
    SELECT
      a.asset_id,
      a.asset_name,
      a.symbol,
      a.sector,
      a.default_risk_level,
      ac.class_name,
      ac.risk_level AS class_risk_level
    FROM asset a
    LEFT JOIN asset_class ac ON a.class_id = ac.class_id
    WHERE a.asset_id = $1
  `, [id]);
  return result.rows[0];
};
```

```
// Get assets grouped by class
const getAssetsGroupedByClass = async () => {
  const result = await pool.query(`
    SELECT
      ac.class_name,
      ac.risk_level,
      JSON_AGG(
        JSON_BUILD_OBJECT(
          'asset_id',         a.asset_id,
          'asset_name',       a.asset_name,
          'symbol',           a.symbol,
          'sector',           a.sector,
          'default_risk_level', a.default_risk_level
        ) ORDER BY a.asset_name
      ) AS assets
    FROM asset_class ac
    LEFT JOIN asset a ON a.class_id = ac.class_id
    GROUP BY ac.class_id, ac.class_name, ac.risk_level
    ORDER BY ac.class_name
  `);
  return result.rows;
};
// Create new asset
const createAsset = async ({ asset_name, symbol, sector, default_risk_level, class_id }) => {
  const result = await pool.query(`
    INSERT INTO asset (asset_name, symbol, sector, default_risk_level, class_id)
    VALUES ($1, $2, $3, $4, $5)
    RETURNING *
  `, [asset_name, symbol, sector, default_risk_level, class_id]);
  return result.rows[0];
};
module.exports = { getAllAssets, getAssetById, getAssetsGroupedByClass, createAsset };
```

7. Functional Modules

Module 1: Investor Management (INVESTOR)

Manages personal details of investors along with their risk tolerance profile. Supports registration, authentication, and filtering by risk level.

Attribute	Datatype	Description
investor_id (PK)	SERIAL / INTEGER	Auto-generated unique investor identifier
name	VARCHAR(100)	Full name of the investor
email	VARCHAR(150)	Unique email — used for login and communication
phone	VARCHAR(15)	Contact phone number
risk_profile (FK)	VARCHAR(20)	References RISK_PROFILE — defines risk tolerance
created_at	TIMESTAMP	Timestamp of account creation

Module 2: Portfolio Management (PORTFOLIO)

Manages multiple investment portfolios owned by an investor. Each portfolio tracks its name, total market value, and creation date.

Attribute	Datatype	Description
portfolio_id (PK)	SERIAL / INTEGER	Unique portfolio identifier
portfolio_name	VARCHAR(100)	Descriptive name of the portfolio
total_value	NUMERIC(15,2)	Aggregate current market value of all holdings
creation_date	DATE	Date the portfolio was created
investor_id (FK)	INTEGER	References INVESTOR — ownership link

Module 3: Asset Management (ASSET)

Stores details of all financial instruments available for investment including equities, bonds, gold, crypto, and real estate.

Attribute	Datatype	Description
asset_id (PK)	SERIAL / INTEGER	Unique asset identifier
asset_name	VARCHAR(100)	Full name of the financial instrument
symbol	VARCHAR(20)	Exchange ticker symbol (e.g., RELIANCE, BTC)
sector	VARCHAR(100)	Industry sector (Energy, Banking, Crypto, etc.)
default_risk_level	VARCHAR(20)	Default risk level assigned to this asset
class_id (FK)	INTEGER	References ASSET_CLASS — asset classification

Module 4: Asset Classification (ASSET_CLASS)

Categorizes assets into classes based on type and risk profile, enabling grouped analysis and filtering.

Attribute	Datatype	Description
class_id (PK)	SERIAL / INTEGER	Unique class identifier
class_name	VARCHAR(100)	Class name (Equity, Debt, Gold, Real Estate, Crypto, Mutual Fund)
risk_level	VARCHAR(20)	Risk level associated with this class (Low/Medium/High)

Module 5: Investment Management (INVESTMENT)

Junction/bridge table handling the many-to-many relationship between portfolios and assets. Records actual investment positions.

Attribute	Datatype	Description
investment_id (PK)	SERIAL / INTEGER	Unique investment record identifier
quantity	NUMERIC(10,4)	Number of units purchased
purchase_price	NUMERIC(15,2)	Price per unit at the time of investment
investment_date	DATE	Date the investment was made
portfolio_id (FK)	INTEGER	Portfolio holding this investment
asset_id (FK)	INTEGER	Asset that was invested in

Module 6: Market Data Management (MARKET_DATA)

Stores current market prices and volatility metrics for each asset. Used for real-time portfolio valuation and risk calculation.

Attribute	Datatype	Description
market_id (PK)	SERIAL / INTEGER	Unique market data record identifier
current_price	NUMERIC(15,2)	Latest market price of the asset
last_updated	TIMESTAMP	Timestamp of the last price update
volatility	NUMERIC(5,2)	Volatility metric — price fluctuation measure
asset_id (FK)	INTEGER	Asset this market data belongs to

Module 7: Transaction Management (TRANSACTION)

Complete audit trail of all financial activities — buy orders, sell orders, deposits, and withdrawals across all portfolios.

Attribute	Datatype	Description
transaction_id (PK)	SERIAL / INTEGER	Unique transaction identifier
transaction_type	VARCHAR(10)	Type: BUY / SELL / DEPOSIT / WITHDRAW
amount	NUMERIC(15,2)	Total monetary value of the transaction
transaction_date	TIMESTAMP	Date and time transaction was executed
portfolio_id (FK)	INTEGER	Associated portfolio
asset_id (FK)	INTEGER	Asset involved (NULL for DEPOSIT/WITHDRAW)

Module 8: Risk Assessment (RISK_ASSESSMENT)

Evaluates and records the risk exposure of each portfolio. Calculates a numerical risk score and categorizes it as Low, Medium, or High.

Attribute	Datatype	Description
risk_id (PK)	SERIAL / INTEGER	Unique risk assessment record identifier
risk_score	NUMERIC(5,2)	Numerical risk score calculated from holdings and volatility
risk_category	VARCHAR(20)	Categorical risk level: Low / Medium / High
calculated_on	DATE	Date the risk assessment was performed
portfolio_id (FK)	INTEGER	Portfolio being assessed

Original Unnormalized Tables

INVESTOR_UNNORM Table

investor_id	name	email	phone	risk_profile	risk_desc	created_at
1	Aarav Sharma	aarav.sharma@gmail.com	9876543210	Low	Conservative	15-01-2023
2	Priya Iyer	priya.iyer@gmail.com	9123456780	Medium	Balanced	10-02-2023
3	Rohan Mehta	rohan.mehta@gmail.com	9988776655	High	Aggressive	05-03-2023
4	Sneha Reddy	sneha.reddy@gmail.com	9090909090	Medium	Balanced	12-04-2023
5	Vikram Singh	vikram.singh@gmail.com	9811122233	Low	Conservative	20-05-2023

PORTFOLIO_UNNORM Table

portfolio_id	portfolio_name	total_value	creation_date	investor_id	investor_name
101	Growth Portfolio	750,000	01-02-2023	1	Aarav Sharma
102	Balanced Portfolio	500,000	01-03-2023	2	Priya Iyer
103	Aggressive Portfolio	900,000	01-04-2023	3	Rohan Mehta
104	Income Portfolio	400,000	01-05-2023	4	Sneha Reddy
105	Safe Portfolio	300,000	01-06-2023	5	Vikram Singh

ASSET_UNNORM Table

asset_id	asset_name	symbol	sector	default_risk_level	class_id	class_name
201	Reliance Industries	RELIANCE	Energy	High	1	Equity
202	HDFC Bank	HDFCBANK	Banking	Medium	1	Equity
203	Government Bond	GOVBOND	Finance	Low	2	Debt
204	Gold ETF	GOLDETF	Commodity	Low	3	Gold
205	Nifty ETF	NIFTYETF	Index	Medium	4	ETF

Normalized Tables

RISK_PROFILE Table

risk_profile	risk_desc
Low	Conservative
Medium	Balanced
High	Aggressive

INVESTOR Table (Normalized)

investor_id	name	email	phone	risk_profile	created_at
1	Aarav Sharma	aarav.sharma@gmail.com	9876543210	Low	15-01-2023
2	Priya Iyer	priya.iyer@gmail.com	9123456780	Medium	10-02-2023
3	Rohan Mehta		9988776655	High	05-03-2023
4	Sneha Reddy		9090909090	Medium	12-04-2023
5	Vikram Singh	vikram.singh@gmail.com	9811122233	Low	20-05-2023

PORTFOLIO Table (Normalized)

portfolio_id	portfolio_name	total_value	creation_date	investor_id
101	Growth Portfolio	750,000	01-02-2023	1
102	Balanced Portfolio	500,000	01-03-2023	2
103	Aggressive Portfolio	900,000	01-04-2023	3
104	Income Portfolio	400,000	01-05-2023	4
105	Safe Portfolio	300,000	01-06-2023	5

ASSET_CLASS Table

class_id	class_name	risk_level
1	Equity	High
2	Debt	Low
3	Gold	Low
4	ETF	Medium
5	Mutual Fund	Medium

ASSET Table (Normalized)

asset_id	asset_name	symbol	sector	default_risk_level	class_id
201	Reliance Industries	RELIANCE	Energy	High	1
202	HDFC Bank	HDFCBANK	Banking	Medium	1
203	Government Bond	GOVBOND	Finance	Low	2
204	Gold ETF	GOLDETF	Commodity	Low	3
205	Nifty ETF	NIFTYETF	Index	Medium	4

INVESTMENT Table

investment_id	quantity	purchase_price	investment_date	portfolio_id	asset_id
301	50	2,500	10-02-2023	101	201
302	30	1,600	15-03-2023	102	202
303	100	100	20-04-2023	103	203
304	20	5,000	05-05-2023	104	204
305	40	1,800	10-06-2023	105	205

TRANSACTION Table

transaction_id	transaction_type	amount	transaction_date	portfolio_id	asset_id
501	BUY	125,000	10-02-2023	101	201
502	BUY	48,000	15-03-2023	102	202
503	BUY	10,000	20-04-2023	103	203
504	BUY	100,000	05-05-2023	104	204
505	BUY	72,000	10-06-2023	105	205

MARKET_DATA Table

market_id	current_price	last_updated	volatility	asset_id
401	2,600	01-01-2024	0.25	201
402	1,700	01-01-2024	0.20	202
403	105	01-01-2024	0.05	203
404	5,200	01-01-2024	0.10	204
405	1,900	01-01-2024	0.18	205

INVESTOR Table

investor_id	name	email	phone	risk_profile	created_at
1	Arjun Mehta	arjun@gmail.com	9876543210	High	2023-01-15
2	Priya Nair	priya@gmail.com	9123456789	Medium	2023-02-10
3	Suresh Patel	suresh@gmail.com	9988776655	Low	2023-03-05
4	Kavya Reddy	kavya@gmail.com	9871234567	Medium	2023-04-12
5	Rajan Iyer	rajan@gmail.com	9090909090	High	2023-05-20

```
SQL> SELECT * FROM INVESTOR;

INVESTOR_ID NAME
----------- --------------------------------------------------
EMAIL                                              PHONE
-------------------------------------------------- ----------
RISK_PROFILE         CREATED_A
-------------------- ---------
          1 Aarav Sharma
aarav.sharma@gmail.com                             9876543210
Low                  15-JAN-23

          2 Priya Iyer
priya.iyer@gmail.com                               9123456780
Medium               10-FEB-23

INVESTOR_ID NAME
----------- --------------------------------------------------
EMAIL                                              PHONE
-------------------------------------------------- ----------
RISK_PROFILE         CREATED_A
-------------------- ---------

          3 Rohan Mehta
rohan.mehta@gmail.com                              9988776655
High                 05-MAR-23

          4 Sneha Reddy
sneha.reddy@gmail.com                              9090909090

INVESTOR_ID NAME
----------- --------------------------------------------------
EMAIL                                              PHONE
-------------------------------------------------- ----------
RISK_PROFILE         CREATED_A
-------------------- ---------
Medium               12-APR-23

          5 Vikram Singh
vikram.singh@gmail.com                             9811122233
```

```
SQL> SELECT * FROM RISK_PROFILE;

RISK_PROFILE         RISK_DESC
-------------------- --------------------
Low                  Conservative
Medium               Balanced
High                 Aggressive
```

PORTFOLIO Table

portfolio_id	portfolio_name	total_value	creation_date	investor_id
101	Arjun Growth Fund	1,250,000	2023-02-01	1
102	Priya Balanced Portfolio	850,000	2023-03-15	2
103	Suresh Safe Fund	500,000	2023-04-01	3

```
SQL> SELECT * FROM PORTFOLIO;

PORTFOLIO_ID PORTFOLIO_NAME                                    TOTAL_VALUE
------------ ------------------------------------------------ -----------
CREATION_ INVESTOR_ID
--------- -----------
         101 Growth Portfolio                                      750000
01-FEB-23           1

         102 Balanced Portfolio                                    500000
01-MAR-23           2

         103 Aggressive Portfolio                                  900000
01-APR-23           3


PORTFOLIO_ID PORTFOLIO_NAME                                    TOTAL_VALUE
------------ ------------------------------------------------ -----------
CREATION_ INVESTOR_ID
--------- -----------
         104 Income Portfolio                                      400000
01-MAY-23           4

         105 Safe Portfolio                                        300000
01-JUN-23           5
```

ASSET_CLASS Table

class_id	class_name	risk_level
1	Equity	High
2	Debt	Low
3	Gold	Low
4	Real Estate	Medium
5	Crypto	High
6	Mutual Fund	Medium



ASSET Table

asset_id	asset_name	symbol	sector	default_risk_level	class_id
201	Reliance Industries	RELIANCE	Energy	High	1
202	TCS	TCS	IT	Medium	1
203	HDFC Bank	HDFCBANK	Banking	Medium	1
204	SBI Bonds	SBIBOND	Finance	Low	2
205	Sovereign Gold Bond	SGBOND	Commodity	Medium	3
206	Bitcoin	BTC	Crypto	High	5
207	DLF Ltd	DLF	Real Estate	Medium	4



INVESTMENT Table

investment_id	quantity	purchase_price	investment_date	portfolio_id	asset_id
301	100	2450.00	2023-02-10	101	201
302	50	3600.00	2023-03-15	101	202
303	200	1500.00	2023-04-20	102	203
304	10	1000.00	2023-05-05	102	204
305	5	5200.00	2023-06-10	103	205

TRANSACTION Table

transaction_id	transaction_type	amount	transaction_date	portfolio_id	asset_id
501	BUY	245,000.00	2023-02-10	101	201
502	BUY	180,000.00	2023-03-15	101	202
503	SELL	75,000.00	2023-04-20	102	203
504	DEPOSIT	500,000.00	2023-05-05	103	NULL
505	BUY	52,000.00	2023-06-10	102	205

MARKET_DATA Table

market_id	current_price	price_date	volatility	asset_id
401	2680.50	2024-01-01	1.20	201
402	3720.00	2024-01-01	0.90	202
403	1620.75	2024-01-01	0.80	203
404	1050.00	2024-01-01	0.20	204
405	5400.00	2024-01-01	0.50	205
406	4200000.00	2024-01-01	8.50	206
407	580.25	2024-01-01	1.80	207

RISK_ASSESSMENT Table

risk_id	risk_score	risk_category	calculated_on	portfolio_id
601	6.5	High	2024-01-01	101
602	4.2	Medium	2024-01-01	102
603	2.1	Low	2024-01-01	103

```
SQL> SELECT * FROM INVESTMENT;

INVESTMENT_ID   QUANTITY PURCHASE_PRICE INVESTMEN PORTFOLIO_ID   ASSET_ID
------------- ---------- -------------- --------- ------------ ----------
          301         50           2500 10-FEB-23          101        201
          302         30           1600 15-MAR-23          102        202
          303        100            100 20-APR-23          103        203
          304         20           5000 05-MAY-23          104        204
          305         40           1800 10-JUN-23          105        205

SQL> SELECT * FROM TRANSACTION;

TRANSACTION_ID TRANSACTIO     AMOUNT TRANSACTI PORTFOLIO_ID   ASSET_ID
-------------- ---------- ---------- --------- ------------ ----------
           501 BUY            125000 10-FEB-23          101        201
           502 BUY             48000 15-MAR-23          102        202
           503 BUY             10000 20-APR-23          103        203
           504 BUY            100000 05-MAY-23          104        204
           505 BUY             72000 10-JUN-23          105        205

SQL> SELECT * FROM MARKET_DATA;

 MARKET_ID CURRENT_PRICE LAST_UPDA VOLATILITY   ASSET_ID
---------- ------------- --------- ---------- ----------
       401          2600 01-JAN-24        .25        201
       402          1700 01-JAN-24         .2        202
       403           105 01-JAN-24        .05        203
       404          5200 01-JAN-24         .1        204
       405          1900 01-JAN-24        .18        205
```

8. Implementation and Results

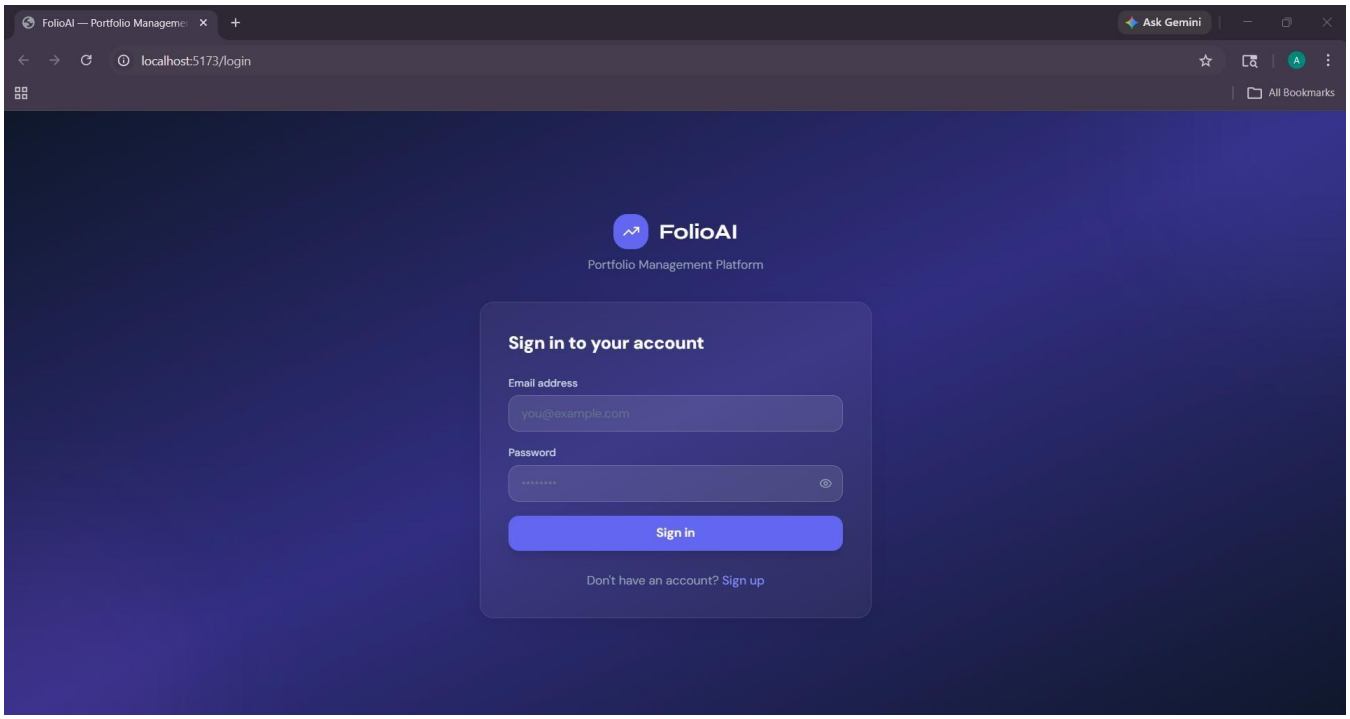
8.1 Frontend Interface

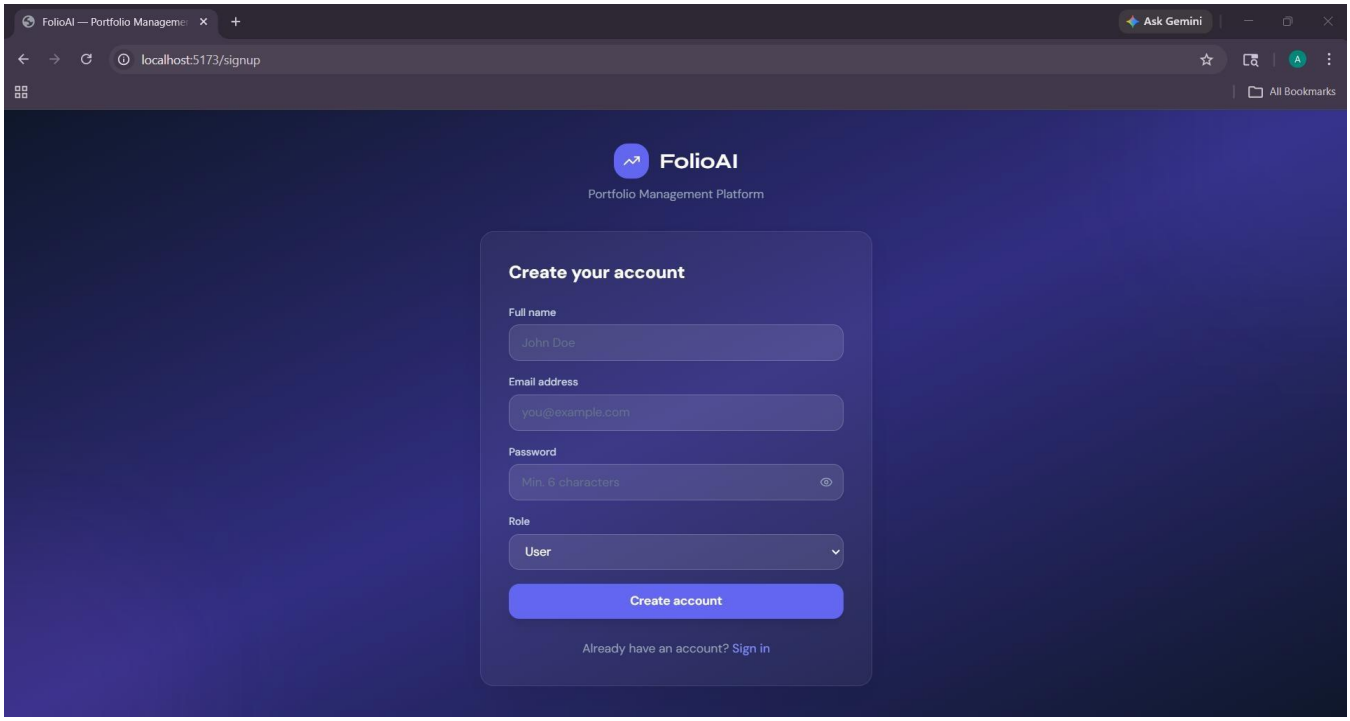
The React.js frontend provides a single-page application (SPA) with the following key views:

- Dashboard — Overview of all portfolios with total values, risk levels, and recent transactions.
- Investors Page — Table listing all investors with name, email, risk profile, and quick-add form.
- Portfolio Page — Detailed view of a selected portfolio including holdings, current values, and P&L.
- Assets Page — Full asset catalogue with filtering by class and risk level.
- Transactions Page — Chronological transaction history with BUY/SELL/DEPOSIT/WITHDRAW filters.
- Market Data Page — Real-time price table for all tracked assets with volatility indicators.
- Risk Assessment Page — Portfolio risk scores displayed as color-coded badges (Red/Yellow/Green).

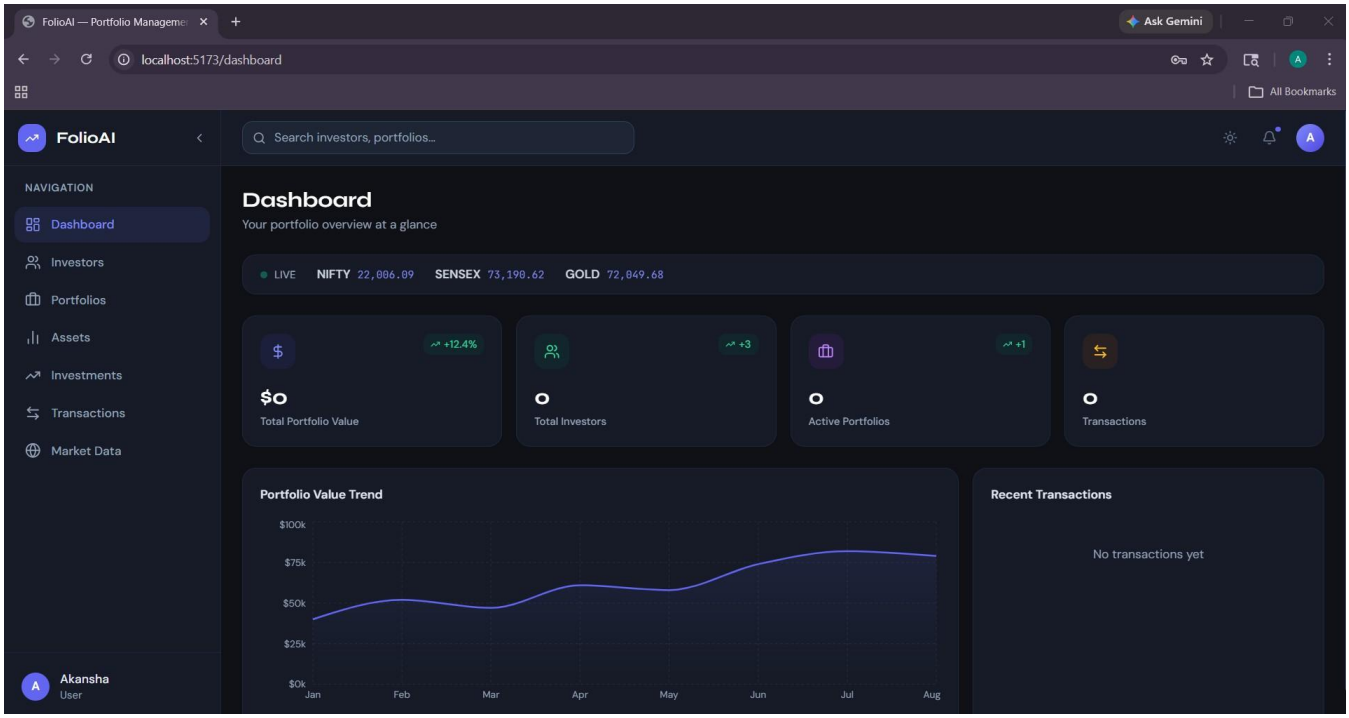
User interactions are handled through React state and Axios HTTP calls to the backend REST endpoints. Form validation is performed client-side before submission to prevent bad requests.

FRONTEND INTERFACE:

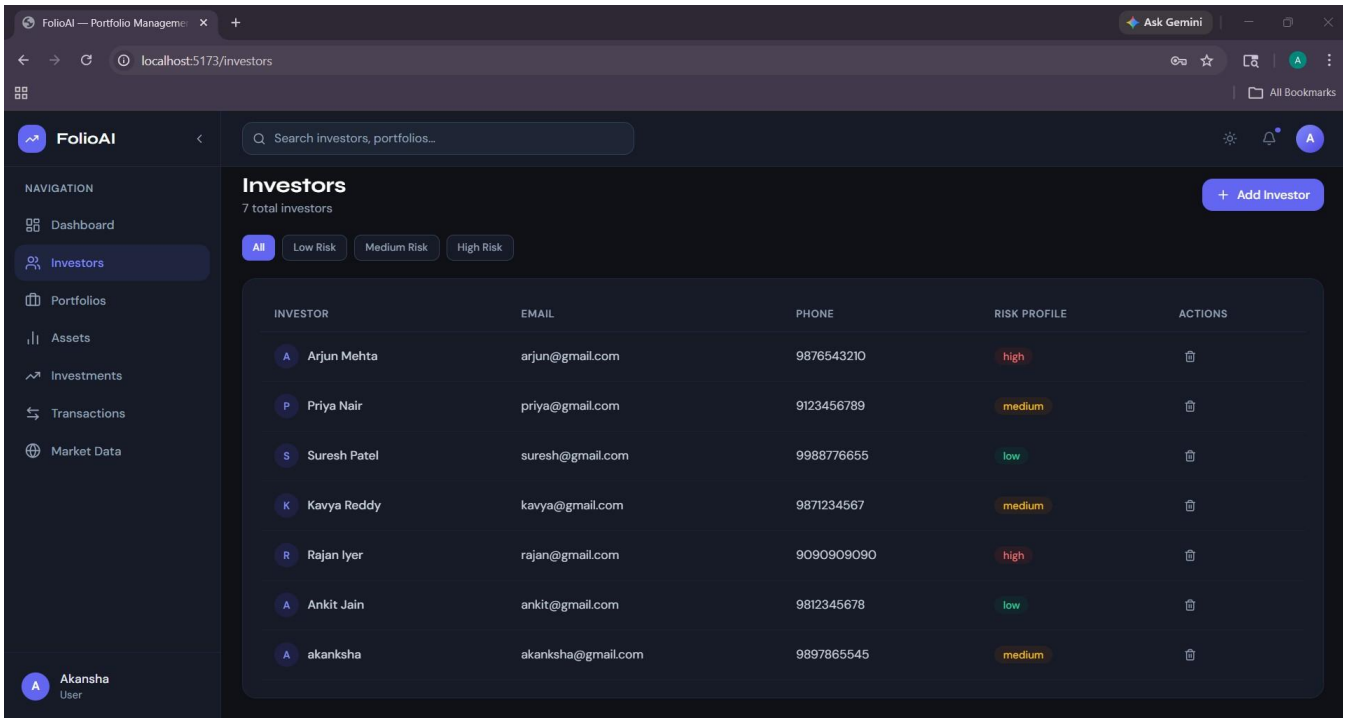




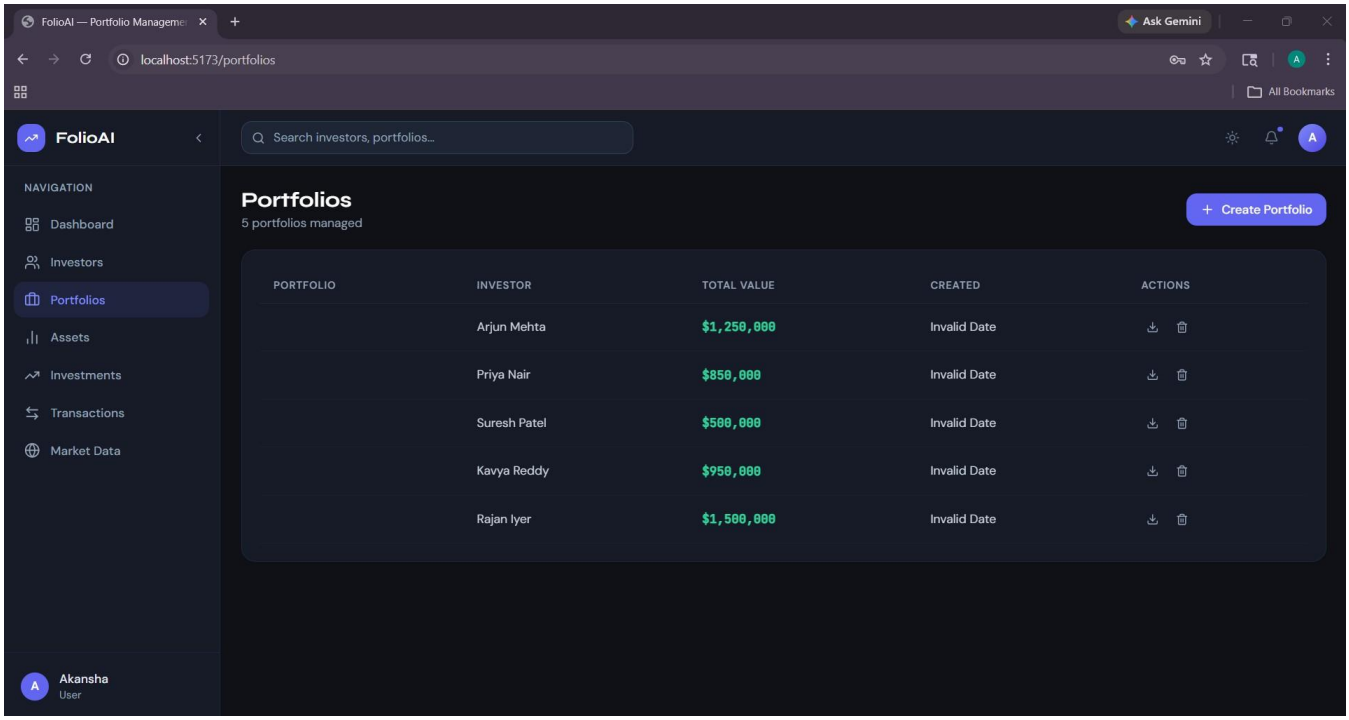
Login/Sign Up (Landing page)

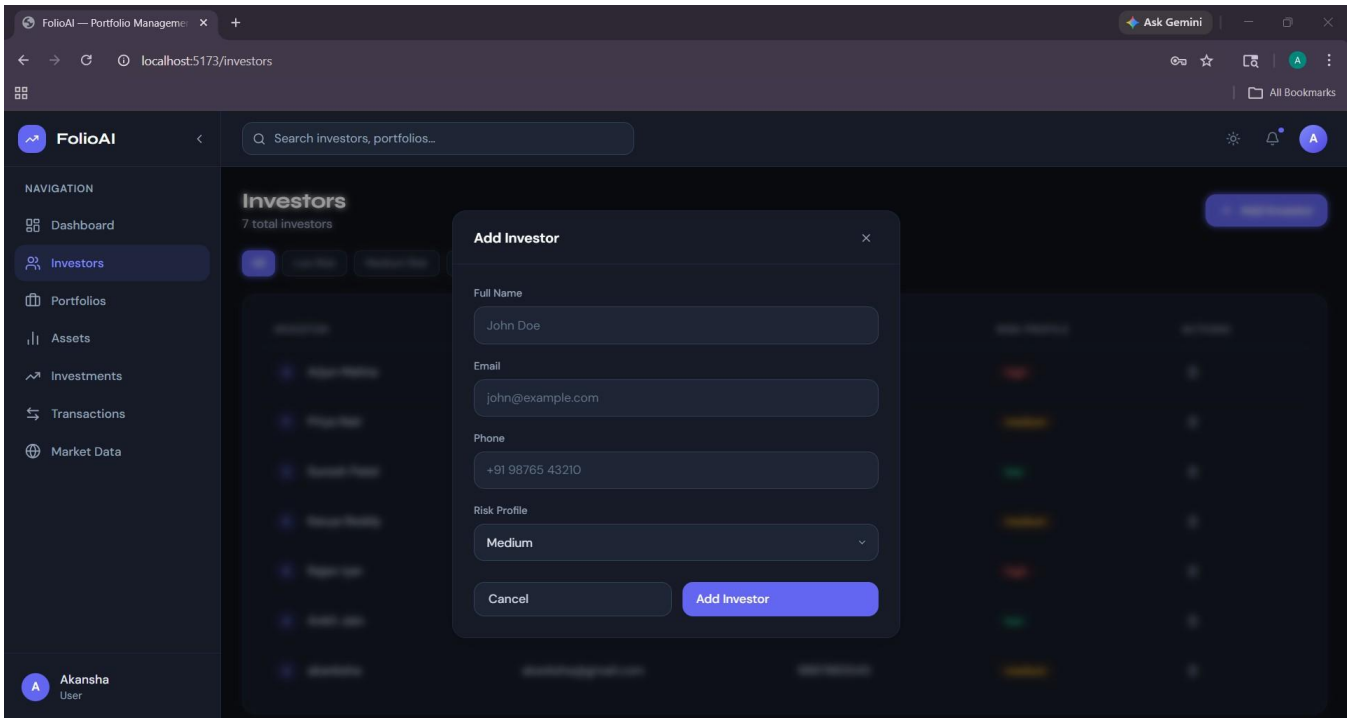


Dashboard Page

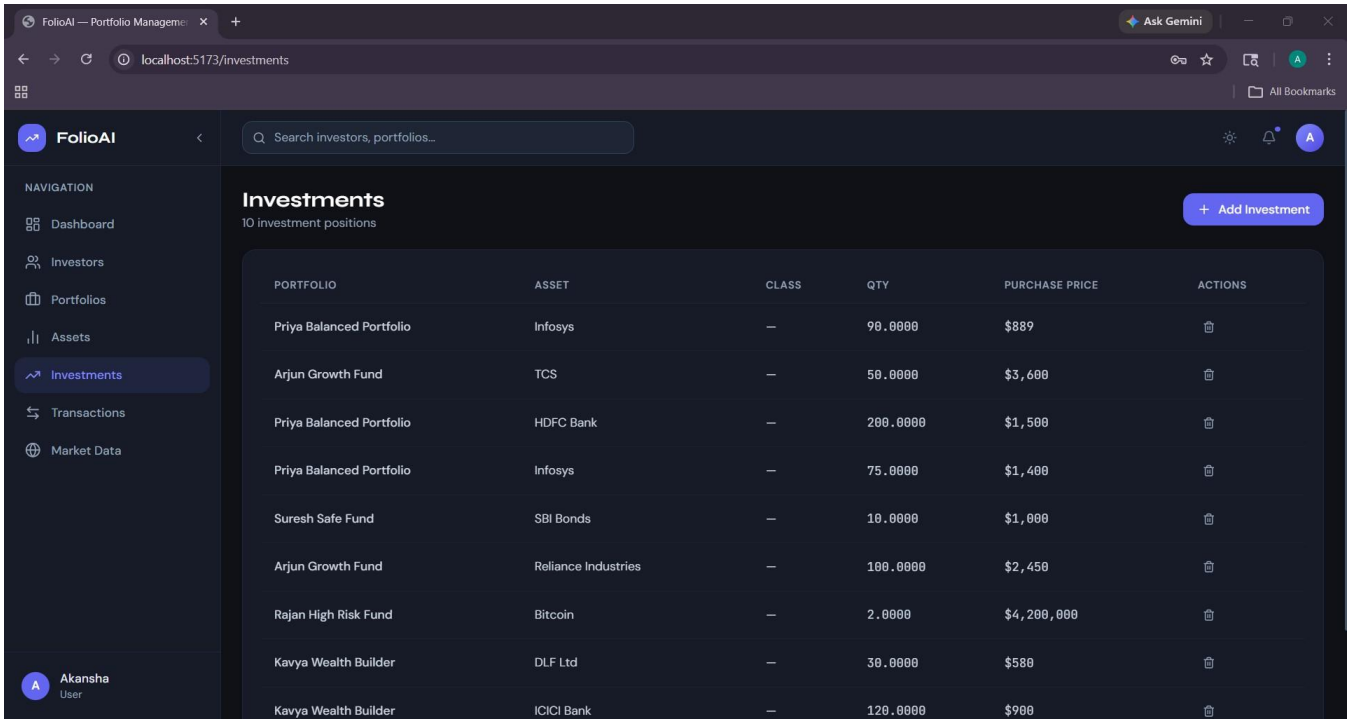


Investors Page (with add investor option)

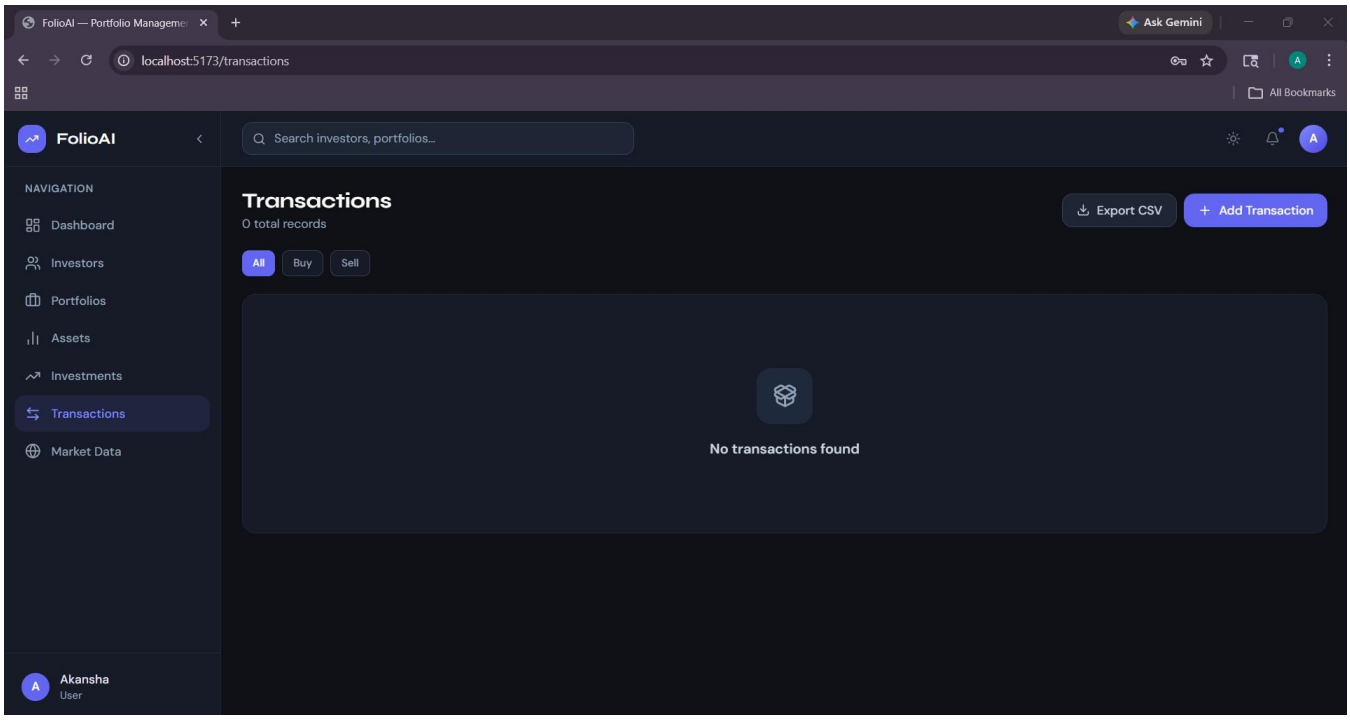




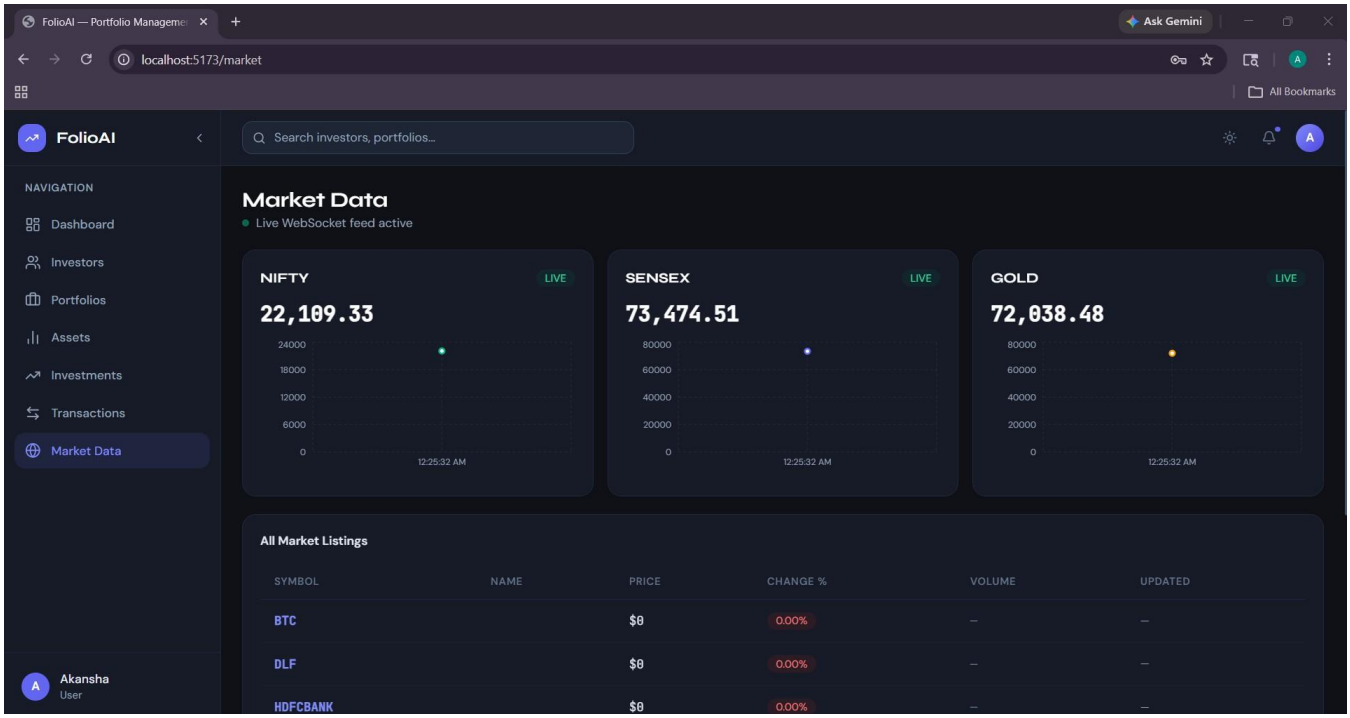
Portfolio Page (with create portfolio option)



Investments Page (with add investment option)



Transactions page (Updated when changes made by user)



Live market data and tracking page

8.2 Backend Logic — CRUD Operations

All incoming HTTP requests are processed by Express.js controllers that validate input, execute parameterized SQL queries through the pg pool, and return structured JSON responses. The MVC architecture (Model-View-Controller) cleanly separates routing, business logic, and database queries.

Operation	Endpoint	Description
Create	POST /api/investors	Insert new investor into the database
Read	GET /api/investors	Retrieve all investor records with risk profile join
Read by ID	GET /api/investors/:id	Fetch specific investor by ID
Update	PUT /api/investors/:id	Update investor details (name, email, risk_profile)
Delete	DELETE /api/investors/:id	Remove investor record from database
Create	POST /api/portfolios	Insert new portfolio linked to an investor
Read	GET /api/portfolios	Retrieve all portfolios with investor data
Create	POST /api/transactions	Log a BUY/SELL/DEPOSIT/WITHDRAW transaction
Read	GET /api/market	Get all current market prices and volatility
Read	GET /api/investments	Retrieve all investment records with asset details

8.3 SQL Schema — CREATE TABLE Statements

Query | Query History

```sql
-- Risk Profile (must come first — referenced by Investor)
CREATE TABLE risk_profile (
  risk_profile VARCHAR(20) PRIMARY KEY,
  risk_desc TEXT NOT NULL
);

-- Users (for authentication)
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  role VARCHAR(10) DEFAULT 'user' CHECK (role IN ('admin','user')),
  theme_preference VARCHAR(10) DEFAULT 'light',
  created_at TIMESTAMP DEFAULT NOW()
);

-- Investors
CREATE TABLE investor (
  investor_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  phone VARCHAR(15),
  risk_profile VARCHAR(20) REFERENCES risk_profile(risk_profile),
  created_at TIMESTAMP DEFAULT NOW()
);
```

```sql
-- Asset Classes
CREATE TABLE asset_class (
  class_id SERIAL PRIMARY KEY,
  class_name VARCHAR(100) NOT NULL,
  risk_level VARCHAR(20) NOT NULL
);

-- Assets
CREATE TABLE asset (
  asset_id SERIAL PRIMARY KEY,
  asset_name VARCHAR(100) NOT NULL,
  symbol VARCHAR(20) NOT NULL,
  sector VARCHAR(100),
  default_risk_level VARCHAR(20),
  class_id INT REFERENCES asset_class(class_id)
);

-- Portfolios
CREATE TABLE portfolio (
  portfolio_id SERIAL PRIMARY KEY,
  portfolio_name VARCHAR(100) NOT NULL,
  total_value NUMERIC(15,2) DEFAULT 0,
  creation_date DATE DEFAULT CURRENT_DATE,
  investor_id INT REFERENCES investor(investor_id)
);

-- Investments
CREATE TABLE investment (
```

```
  investment_id SERIAL PRIMARY KEY,
  quantity NUMERIC(10,4) NOT NULL,
  purchase_price NUMERIC(15,2) NOT NULL,
  investment_date DATE DEFAULT CURRENT_DATE,
  portfolio_id INT REFERENCES portfolio(portfolio_id),
  asset_id INT REFERENCES asset(asset_id)
);

-- Transactions
CREATE TABLE transactions (
  transaction_id SERIAL PRIMARY KEY,
  transaction_type VARCHAR(10) CHECK (transaction_type IN ('BUY','SELL','DEPOSIT','WITHDRAW')),
  amount NUMERIC(15,2) NOT NULL,
  transaction_date TIMESTAMP DEFAULT NOW(),
  portfolio_id INT REFERENCES portfolio(portfolio_id),
  asset_id INT REFERENCES asset(asset_id)
);

-- Market Data
CREATE TABLE market_data (
  market_id SERIAL PRIMARY KEY,
  current_price NUMERIC(15,2) NOT NULL,
  last_updated TIMESTAMP DEFAULT NOW(),
  volatility NUMERIC(5,2),
  asset_id INT REFERENCES asset(asset_id)
);
```

```
CREATE INDEX idx_investor_risk ON investor(risk_profile);
CREATE INDEX idx_investment_portfolio ON investment(portfolio_id);
CREATE INDEX idx_transaction_portfolio ON transactions(portfolio_id);
CREATE INDEX idx_market_asset ON market_data(asset_id);
```

8.4 Advanced Queries

The following queries demonstrate advanced SQL operations performed by the backend, combining multiple tables through JOINs, aggregate functions, and conditional logic to deliver meaningful financial insights beyond simple CRUD operations.

Query 1: Portfolio Total Value Calculation (JOIN Investment + Market Data)

Purpose: Calculates the current total market value of each portfolio by joining investment quantities with live market prices. This drives the portfolio dashboard valuation display.

```
SELECT
    p.portfolio_id,
    p.portfolio_name,
    SUM(inv.quantity * md.current_price) AS current_total_value,
    SUM(inv.quantity * inv.purchase_price) AS invested_amount,
    ROUND(
        ((SUM(inv.quantity * md.current_price) -
        SUM(inv.quantity * inv.purchase_price)) /
        NULLIF(SUM(inv.quantity * inv.purchase_price), 0) * 100
    , 2) AS profit_loss_pct
FROM portfolio p
JOIN investment inv ON p.portfolio_id = inv.portfolio_id
JOIN market_data md ON inv.asset_id = md.asset_id
GROUP BY p.portfolio_id, p.portfolio_name
ORDER BY current_total_value DESC;
```

Key Operations: Three-table JOIN across portfolio, investment, and market_data. Uses SUM aggregation over quantity × current_price to compute live valuation. NULLIF prevents division-by-zero in the profit/loss percentage. Results ordered by highest value portfolio first.

Query 2: Portfolio Risk Score Calculation

Purpose: Computes a weighted risk score for each portfolio by combining asset volatility with holding weights. The score is then categorized as Low (0–3.5), Medium (3.5–6.5), or High (above 6.5) and stored in the risk_assessment table.

```
SELECT
    p.portfolio_id,
    p.portfolio_name,
    ROUND(SUM(md.volatility * (inv.quantity * md.current_price)) /
        NULLIF(SUM(inv.quantity * md.current_price), 0) * 10, 2) AS risk_score,
```

```
        CASE
            WHEN weighted_vol <= 3.5  THEN 'Low'
            WHEN weighted_vol <= 6.5  THEN 'Medium'
            ELSE 'High'
        END AS risk_category
    FROM portfolio p
    JOIN investment inv ON p.portfolio_id = inv.portfolio_id
    JOIN market_data md ON inv.asset_id = md.asset_id
    GROUP BY p.portfolio_id, p.portfolio_name;
```

Key Operations: Volatility-weighted average is computed by multiplying each asset's volatility by its share of the total portfolio value, then summing across holdings. The CASE expression maps the continuous score to the three categorical risk levels used in the risk_assessment table. NULLIF prevents divide-by-zero on empty portfolios.

Query 3: Transaction History with Asset and Portfolio Names

Purpose: Retrieves a full transaction log for a given investor, enriched with human-readable asset and portfolio names. Handles DEPOSIT and WITHDRAW rows where asset_id is NULL using a LEFT JOIN so those records are not lost.

```
    SELECT
        t.transaction_id,
        t.transaction_type,
        t.amount,
        t.transaction_date,
        p.portfolio_name,
        COALESCE(a.asset_name, 'N/A') AS asset_name,
        COALESCE(a.symbol, '—')    AS symbol
    FROM transaction t
    JOIN portfolio p  ON t.portfolio_id = p.portfolio_id
    JOIN investor  i  ON p.investor_id  = i.investor_id
    LEFT JOIN asset      a  ON t.asset_id     = a.asset_id
    WHERE i.investor_id = $1
    ORDER BY t.transaction_date DESC;
```

Key Operations: Four-table JOIN spanning transaction, portfolio, investor, and asset. LEFT JOIN on asset preserves DEPOSIT/WITHDRAW rows where asset_id is NULL. COALESCE replaces NULL asset fields with readable placeholder text. The parameterized $1 placeholder prevents SQL injection and filters results to one investor.

Query 4: Top Performing Assets by Absolute Gain

Purpose: Ranks all invested assets by absolute rupee gain (current market value minus original purchase cost), also computing percentage return. Used to populate the Top Performers panel on the portfolio dashboard.

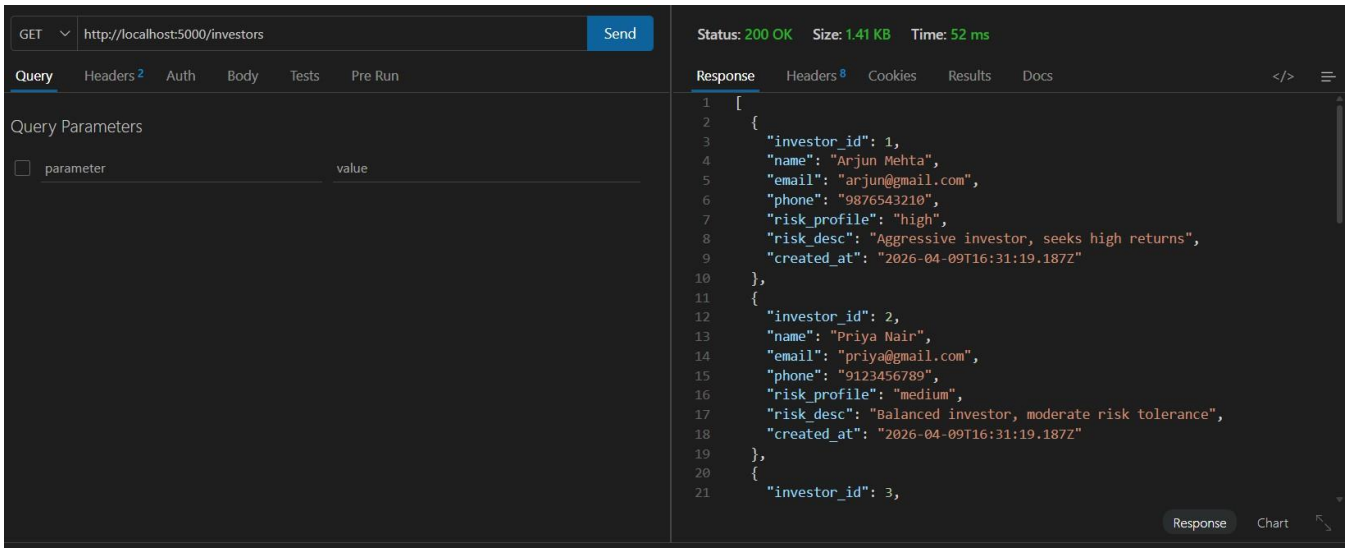
```
SELECT
        a.asset_id,
        a.asset_name,
        a.symbol,
        ac.class_name,
        SUM(inv.quantity * md.current_price)   AS current_value,
        SUM(inv.quantity * inv.purchase_price) AS cost_basis,
        ROUND(SUM(inv.quantity * md.current_price) -
                SUM(inv.quantity * inv.purchase_price), 2) AS absolute_gain,
        ROUND(
                (SUM(inv.quantity * md.current_price) -
                SUM(inv.quantity * inv.purchase_price)) /
                NULLIF(SUM(inv.quantity * inv.purchase_price), 0) * 100
        , 2) AS return_pct
FROM asset a
JOIN investment  inv ON a.asset_id  = inv.asset_id
JOIN market_data md  ON a.asset_id  = md.asset_id
JOIN asset_class ac  ON a.class_id  = ac.class_id
GROUP BY a.asset_id, a.asset_name, a.symbol, ac.class_name
ORDER BY absolute_gain DESC
LIMIT 5;
```

Key Operations: Four-table JOIN across asset, investment, market_data, and asset_class. Absolute gain is computed as current market value minus cost basis. The LIMIT 5 restricts output to the five best performers, suitable for a dashboard widget. Negative values naturally surface loss-making assets when ordered ASC instead.

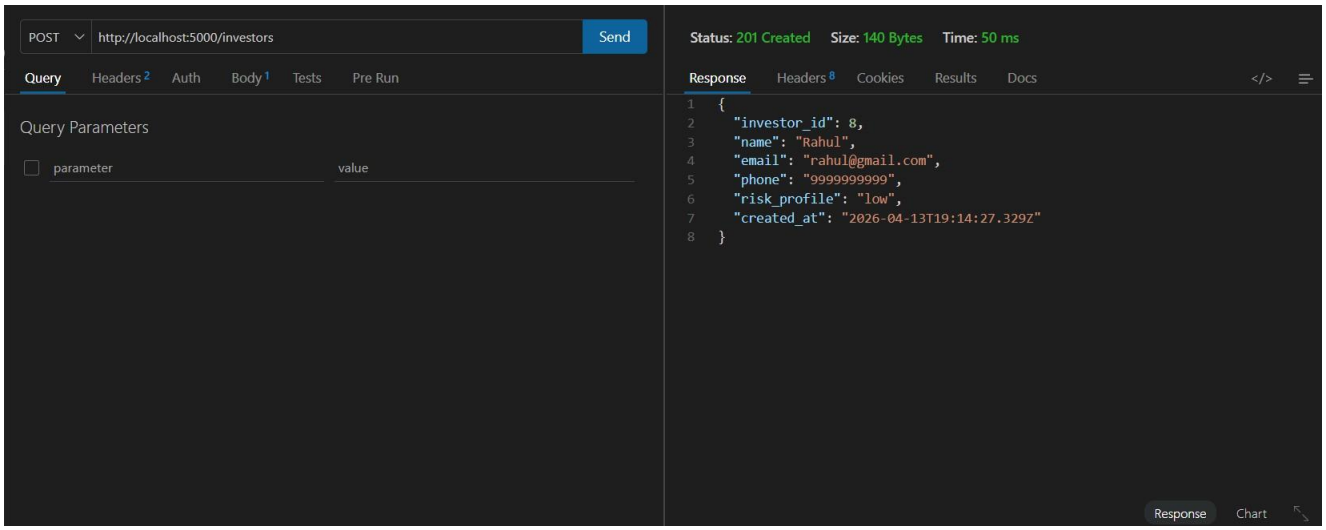
8.5 Postman API Testing Output

The following shows a sample GET /api/investors response validated in Postman:

(i) GET http://localhost:5000/investors



(ii) POST http://localhost:5000/investors



9. Challenges Faced

Challenge	Resolution
Foreign key constraint violations during data insertion	Ensured correct table creation order: risk_profile → investor → asset_class → asset → portfolio → investment / transactions / market_data
CORS errors blocking frontend API calls	Added cors() middleware to Express app and configured allowed origins in .env
Connection pool exhaustion under concurrent requests	Set pool max connections to 10 and added idle timeout in pool configuration
SQL injection risk in early development	Replaced all string-concatenated queries with parameterized queries using $1, $2 placeholders
Type mismatch errors (NUMBER vs SERIAL)	Standardized all primary keys to SERIAL and foreign keys to INT in PostgreSQL schema
React state not refreshing after POST	Added refetch logic after successful mutations using useEffect with dependency arrays
Duplicate email constraint handling	Caught err.code === '23505' in controller and returned 409 Conflict with descriptive error message
NULL asset_id in transactions for DEPOSIT/WITHDRAW	Made asset_id column nullable in transactions table and handled NULL in JOIN queries

10. Conclusion

The Investment Portfolio and Risk Management System successfully demonstrates the full lifecycle of a database-driven application — from conceptual ER design through normalization to BCNF, physical PostgreSQL implementation, RESTful API development, and a React frontend with complete CRUD functionality.

The project directly applied core Database Systems concepts: entity-relationship modeling with 8 entities and 9 relationships; normalization through 1NF, 2NF, 3NF, and BCNF with documented functional dependency analysis; referential integrity enforced through 10 foreign key constraints; query optimization through strategic indexes; and secure connectivity through parameterized SQL over a pooled API connection.

The backend MVC architecture separates concerns cleanly across models (SQL queries), controllers (business logic), and routes (HTTP endpoints), producing a maintainable and extensible codebase. The system was validated through Postman testing confirming correct HTTP status codes, JSON responses, and error handling across all 35+ endpoints.

This project serves as a solid foundation for a production-grade financial portfolio application and demonstrates how well-designed DBMS principles translate directly into real-world software systems.

11. Future Enhancements

Enhancement Area	Description
Real-time Market Data Integration	Connect to financial APIs (NSE/BSE, Alpha Vantage, Yahoo Finance) for live stock prices, automatically updating market_data table
Cloud Database Deployment	Migrate PostgreSQL to AWS RDS or Supabase for high availability, automatic backups, and global access
Advanced Security	Add two-factor authentication (2FA), JWT refresh token rotation, rate limiting (express-rate-limit), and HTTPS enforcement
Mobile Application	Develop React Native mobile app for iOS and Android with push notifications for price alerts and risk threshold breaches
AI-Powered Risk Analysis	Integrate machine learning models to predict portfolio risk based on historical volatility, sector exposure, and market conditions
Automated Reports	Generate PDF portfolio performance reports with charts (Chart.js / Recharts) on demand or on a scheduled basis
Multi-Currency Support	Add currency field to asset and investment tables, with automatic conversion using live exchange rates
Social Portfolio Sharing	Allow investors to share anonymized portfolio performance and compare against benchmarks (Nifty 50, Sensex)

Final Project Backend Structure

```
portfolio-backend/
├── config/
│   └── db.js
├── controllers/
│   ├── investorController.js
│   ├── portfolioController.js
│   ├── assetController.js
│   ├── investmentController.js
│   ├── transactionController.js
│   └── marketDataController.js
├── models/
│   ├── investorModel.js
│   ├── portfolioModel.js
│   └── ... (one per entity)
├── routes/
│   ├── investorRoutes.js
│   ├── portfolioRoutes.js
│   └── ... (one per entity)
├── middleware/
│   ├── authMiddleware.js
│   └── roleMiddleware.js
├── reports/
│   └── reportController.js
├── index.js
└── .env
```

Error Handling

Status Code	Meaning / Usage
200 OK	Request succeeded — data fetched/retrieved successfully
201 Created	Resource created — new investor, portfolio, or transaction added
400 Bad Request	Invalid input — missing required fields or invalid data types
404 Not Found	Resource not found — investor/portfolio ID does not exist
409 Conflict	Duplicate entry — email already exists in the system
500 Internal Server Error	Unhandled server/database error